# INTERNSHIP-II

AN INTERNSHIP REPORT
submitted to
**COCHIN UNIVERSITY OF SCIENCE & TECHNOLOGY**
by
**SHIVA SAJAY (20423092)**

in partial fulfillment for the award of the degree
of
**DIVISION OF INFORMATION TECHNOLOGY**


SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE & TECHNOLOGY
**KOCHI- 682 022**
**OCTOBER 2025**

# Acknowledgement

I would like to express my sincere gratitude to Dr. Abdulla P, Principal, for providing excellent facilities and encouragement for the successful completion and presentation of my internship. My deepest thanks go to Prof. Santhosh Kumar M.B., Head of IT Department, and Dr. Binsu C. Kovoor, Course Coordinator, for their invaluable guidance, motivation, and continuous support throughout this internship. I would also thank my friends and family for their cooperation, encouragement, and understanding which greatly helped me during this period.

**Shiva Sajay**

# Abstract

This report documents the comprehensive internship project focused on developing an Analytics API to collect and process user interaction data efficiently. The API was built using FastAPI with a PostgreSQL database integrated with TimescaleDB extensions to optimize time-series data handling. The entire system was containerized using Docker to allow easy, portable, and consistent deployment across various environments. The project emphasized scalable data ingestion, flexible aggregation over different time frames, secure deployment on Railway cloud, and simplifying the development-to-deployment lifecycle. This internship enhanced my technical skills in backend API development, database optimization, container orchestration, and cloud deployment.

**Keywords**: FastAPI, PostgreSQL, TimescaleDB, Docker, API development, Data Aggregation.

# Contents

# Chapter 1: Introduction

The internship was undertaken with a focus on developing a modern backend analytics API that can reliably ingest, aggregate, and serve user interaction data for analytical insights. Today's data-driven applications rely heavily on robust analytics to improve user experience, make informed decisions, and optimize performance. However, existing third-party analytics solutions present limitations around data privacy and control. This motivated the creation of an in-house solution that balances functionality, scalability, and deployment ease.

**Objective**: Design an API capable of collecting large volumes of time-series user interaction data, store them efficiently, and allow flexible queries for aggregated insights. To ensure easy deployment and consistent operations across various platforms, containerization with Docker was implemented.

This report details the technical challenges, the chosen software stack, development practice, and deployment methodology. The final solution enables organizations to collect rich user analytics privately with high confidence and simplified infrastructure management.

# Chapter 2: About the Training Platform / Tools Used

- **FastAPI**: Selected for its modern async design, ease of writing RESTful APIs in Python, auto-generated OpenAPI docs, and excellent performance.

- **PostgreSQL with TimescaleDB**: TimescaleDB extends PostgreSQL by adding time-series optimized capabilities like hypertables, facilitating efficient storage and querying of large streams of temporal data like user interaction logs.

- **Docker & Docker Compose**: Used to containerize the API and database. Provided significant benefits including environment consistency, portability, simplified deployment processes, and easier collaboration.

- **Railway Cloud platform**: Used for deploying the Docker containers in a cloud environment to demonstrate real-world usage scenarios.

- **Pydantic and SQLModel**: For defining data models with validation and simplifying database interactions with Pythonic code.
- **Jupyter Notebooks**: Used extensively for testing API endpoints, rapid prototyping, and interactive data exploration.

Each tool helped address specific project requirements while building a scalable, maintainable, and production-ready system.

# Chapter 3: Project Requirements and Planning

**Functional Requirements**:

- Develop an API to ingest user interaction data (such as page views, clicks with timestamps) in real-time.

- Store data in a database optimized for time-series data.
- Allow data aggregation (e.g., per hour, day, week) and queries filtered by page or event type.
- Provide endpoint responses in JSON format with aggregated statistics.
- Secure and limit API access.

**Non-Functional Requirements**:

- The API must be easy to deploy, reproducible, and environment-independent.
- Fast response times for aggregated queries over large datasets.
- Scalable design to handle increasing volumes of data and analytical load.
- Maintainable and extensible codebase with clear module separation.

**Planning**:

- Design API endpoints and schema for ingestion and querying.
- Setup TimescaleDB hypertables for efficient storage.
- Plan Dockerfiles and Compose configurations for API and DB containers.
- Break down development into modules and tasks with timelines.

# Chapter 4: Implementation

- FastAPI application with routes for data ingestion, aggregation, and querying.

- Data models defined via Pydantic for validation and SQLModel for database ORM.

- PostgreSQL database with TimescaleDB installed, hypertables used for user interaction data.

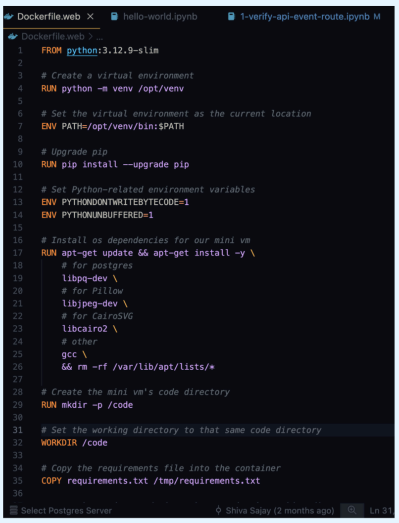
- Aggregation logic implemented using TimescaleDB's time_bucket function in SQL queries for flexible time interval grouping.

- Dockerfile created to containerize API code with Python environment and dependencies.

- Docker Compose file created to orchestrate the API and PostgreSQL service with shared networks and volume mounts for database persistence.
- Secured sensitive configuration such as DB credentials with environment variables.
- Logging and error handling added for robustness.

Challenges included learning Dockerfile syntax, debugging container network issues, optimizing SQL queries, and managing large Docker images.

**Dockerfile**



```
FROM python:3.12.9-slim

# Create a virtual environment
RUN python -m venv /opt/venv

# Set the virtual environment as the current location
ENV PATH=/opt/venv/bin:$PATH

# Upgrade pip
RUN pip install --upgrade pip

# Set Python-related environment variables
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Install os dependencies for our mini vm
RUN apt-get update && apt-get install -y \
    # for postgres
    libpq-dev \
    # for Pillow
    libjpeg-dev \
    # for CairoSVG
    libcairo2 \
    # other
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Create the mini vm's code directory
RUN mkdir -p /code

# Set the working directory to that same code directory
WORKDIR /code

# Copy the requirements file into the container
COPY requirements.txt /tmp/requirements.txt
```

**compose.yaml**

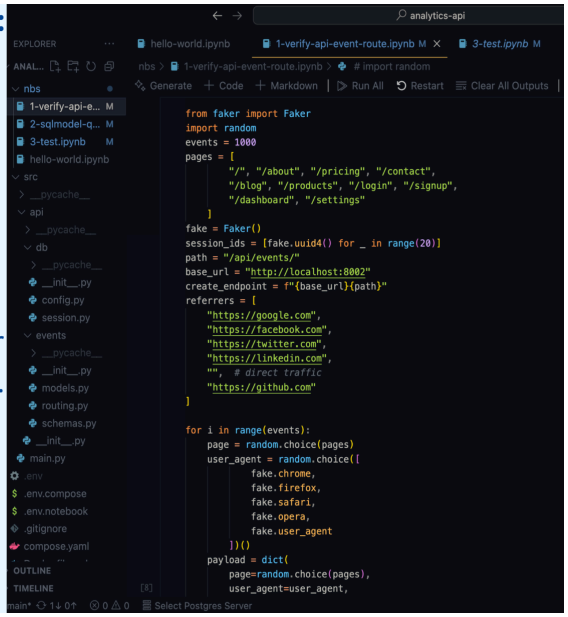
```
services:
  analytics_api:
    image: analytics-api:v1
    build:
      context: .
      dockerfile: Dockerfile.web
    env_file:
      - .env.compose
    ports:
      - "8002:8002"
    command: uvicorn main:app --host 0.0.0.0 --port 8002 --reload
    volumes:
      - ./src:/code:rw
    depends_on:
      db_service:
        condition: service_healthy
    environment:
      - PYTHONPATH=/code
    develop:
      watch:
        - path: Dockerfile.web
          action: rebuild
        - path: requirements.txt
          action: rebuild
        - path: compose.yaml
          action: rebuild

  db_service:
    image: timescale/timescaledb:latest-pg17
    volumes:
      - timescaledb_data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    expose:
```

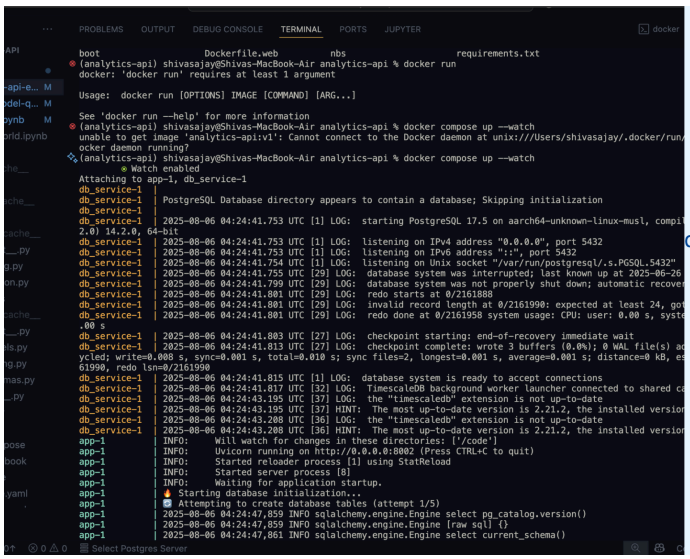
# Chapter 5: Testing and Deployment

- API endpoints tested with Jupyter Notebooks using HTTP requests to validate ingestion and query functionality.
- Unit testing for individual components (models, validation).
- Run Docker Compose locally imitating the production environment to ensure environment parity.
- Deployment on Railway cloud by pushing Docker images and using Docker Compose for multi-container orchestration.
- Setup private networking on Railway to securely expose the API only to trusted services.
- Monitored logs and verified data persistence across server restarts and deployments.

**Other Supporting Tools & Libraries:**

- **Requests, HTTPX, Faker**: Python libraries used for making API calls, generating fake data, and testing.
- **PopSQL**: A collaborative SQL editor for database interaction and verification.
- **Uvicorn & Gunicorn**: Servers for running the FastAPI application.
- **Python Decouple**: A library for managing environment variables.
- **psycopg2**: A PostgreSQL adapter for Python.



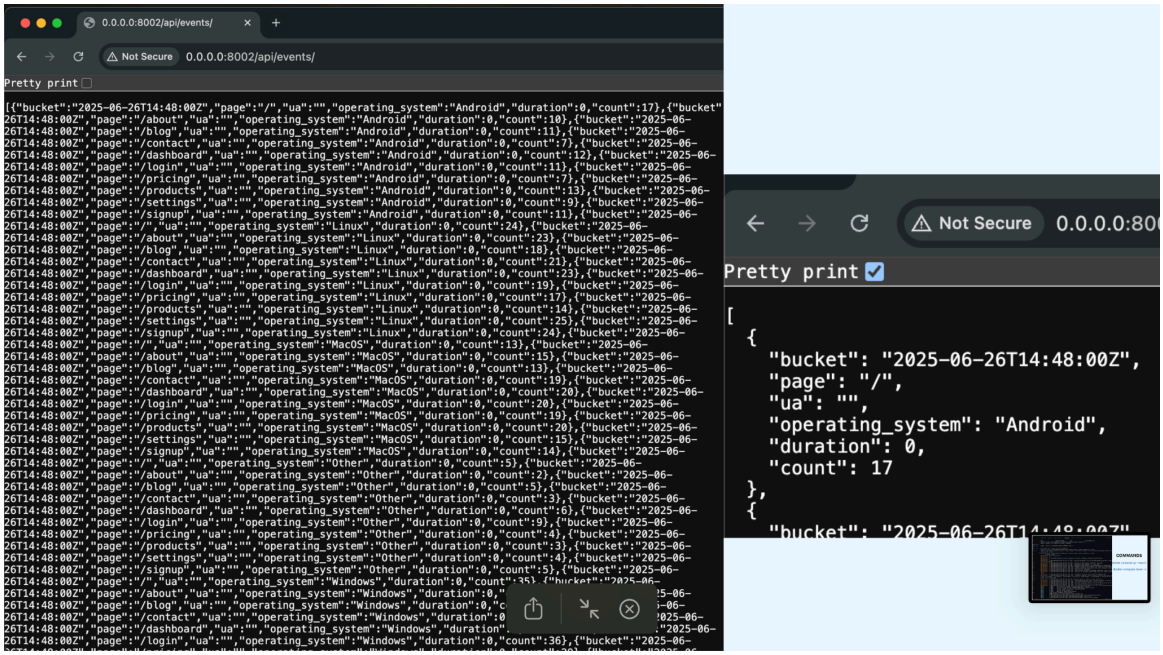



# Chapter 6: Project Outcome and Learnings

- Successfully developed an analytics API capable of ingesting, storing, and querying large-scale user interaction data.
- Learned best practices for API design using FastAPI and asynchronous programming in Python.
- Gained profound understanding of time-series data management with TimescaleDB.
- Mastered Docker containerization including multi-service orchestration, environment consistency, and deployment automation.
- Exposure to cloud deployment concepts and securing microservices.

- Experienced real-world challenges in debugging, optimization, and collaboration using containerized environments.

- Built a valuable foundation for future projects in data-intensive backend services and DevOps.




# Chapter 7: Conclusion and Future Scope

This internship provided a comprehensive exposure to building scalable, maintainable backend data services incorporating modern deployment practices. The designed analytics API achieves the intended objectives of consistent deployment, scalable ingestion, and flexible querying. The skills acquired improve capability to deliver real-world production software.

**Future scope can include:**

- Adding authentication and role-based access control to secure APIs further.
- Enhancing aggregation with real-time streaming data processing using Kafka or RabbitMQ.
- Incorporating a frontend dashboard for visualizing analytics.
- Scaling the database horizontally for extremely large datasets.
- Integrating advanced AI/ML models to predict user behavior based on interaction data.

# References

Include official documentation and tutorials you referred to, such as:

- FastAPI official docs
- PostgreSQL and TimescaleDB manuals
- Docker and Docker Compose documentation
- Railway deployment guides
- Python Pydantic and SQLModel libraries